

Spotting Accuracy Issues in Profile-Guided Optimization

A new methodology to spot metadata propagation errors within optimization pipelines

Supervisor

Prof. Daniele Cono D'Elia

Co-supervisor

Dr. Cristian Assaiante



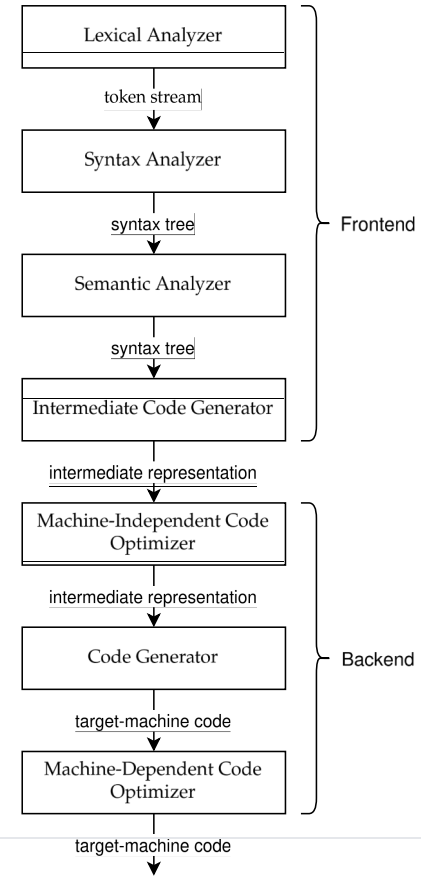
SAPIENZA
UNIVERSITÀ DI ROMA

Table of Contents

1. Compilers
2. Optimizations
3. Profile Guided Optimization
4. The LLVM Project
5. Profile Guided Optimization in LLVM
6. Profile Information Propagation
7. Problem Formalization
8. Optimization Phase
9. Search Phase
10. Results

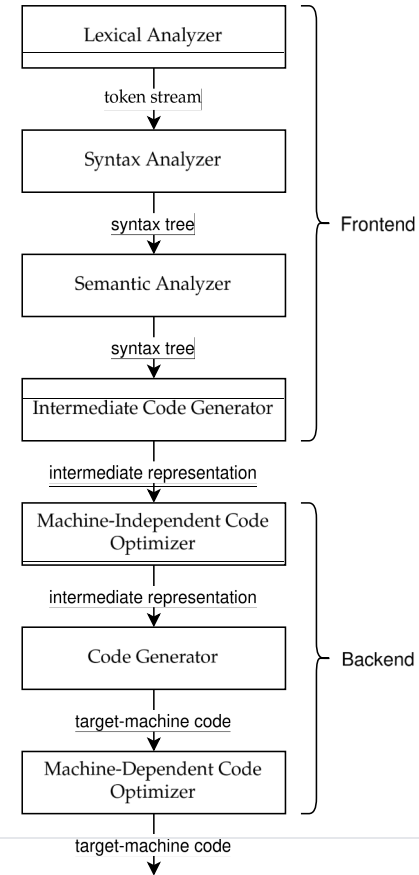


Compilers



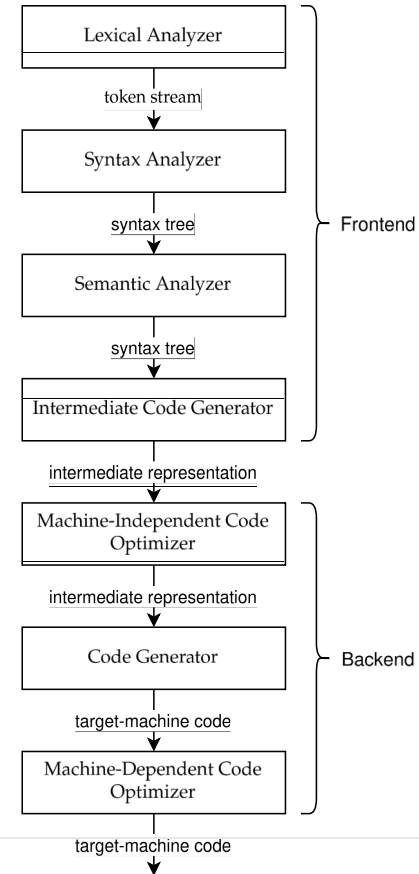
Compilers

- What is a compiler?
 - Translates source code -> target code
 - Preserves program semantic
 - *Optimizes* code during translation



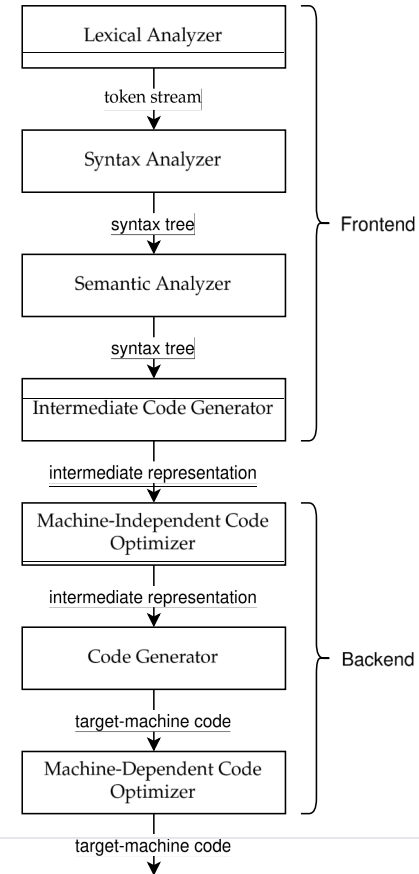
Compilers

- What is a compiler?
 - Translates source code -> target code
 - Preserves program semantic
 - *Optimizes* code during translation
- Compiler Structure (3 Stages)
 - Front-end -> Parses source code -> builds IR
 - Middle-end -> Optimizes IR
 - Back-end -> Generates target code



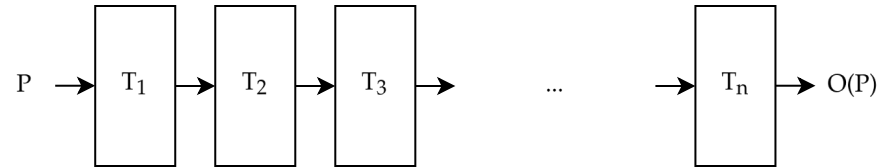
Compilers

- What is a compiler?
 - Translates source code -> target code
 - Preserves program semantic
 - *Optimizes* code during translation
- Compiler Structure (3 Stages)
 - Front-end -> Parses source code -> builds IR
 - Middle-end -> Optimizes IR
 - Back-end -> Generates target code
- Intermediate Representation (IR)
 - Multiple IR types exist
 - Depends on compiler design goals



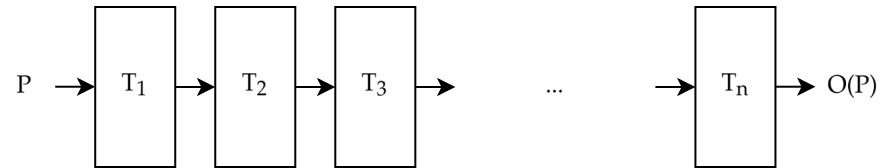
Optimizations

- The optimization process
 - A sequence of transformations (a.k.a. Pipeline)
 - The IR is processed at each pass
 - Output is a *better* program w.r.t. some performance metrics.
 - Supported by static or dynamic analysis that estimate control-flow



Optimizations

- The optimization process
 - A sequence of transformations (a.k.a. Pipeline)
 - The IR is processed at each pass
 - Output is a *better* program w.r.t. some performance metrics.
 - Supported by static or dynamic analysis that estimate control-flow



Profile Guided Optimization



Profile Guided Optimization

- A smart optimization technique
 - Leverages *control flow information* captured at runtime to steer optimizations
 - Notable performance improvement



Profile Guided Optimization

- A smart optimization technique
 - Leverages *control flow information* captured at runtime to steer optimizations
 - Notable performance improvement
- Control flow information (or *profile*) a.k.a.
 - Weights on control flow edges
 - Counts on basic blocks

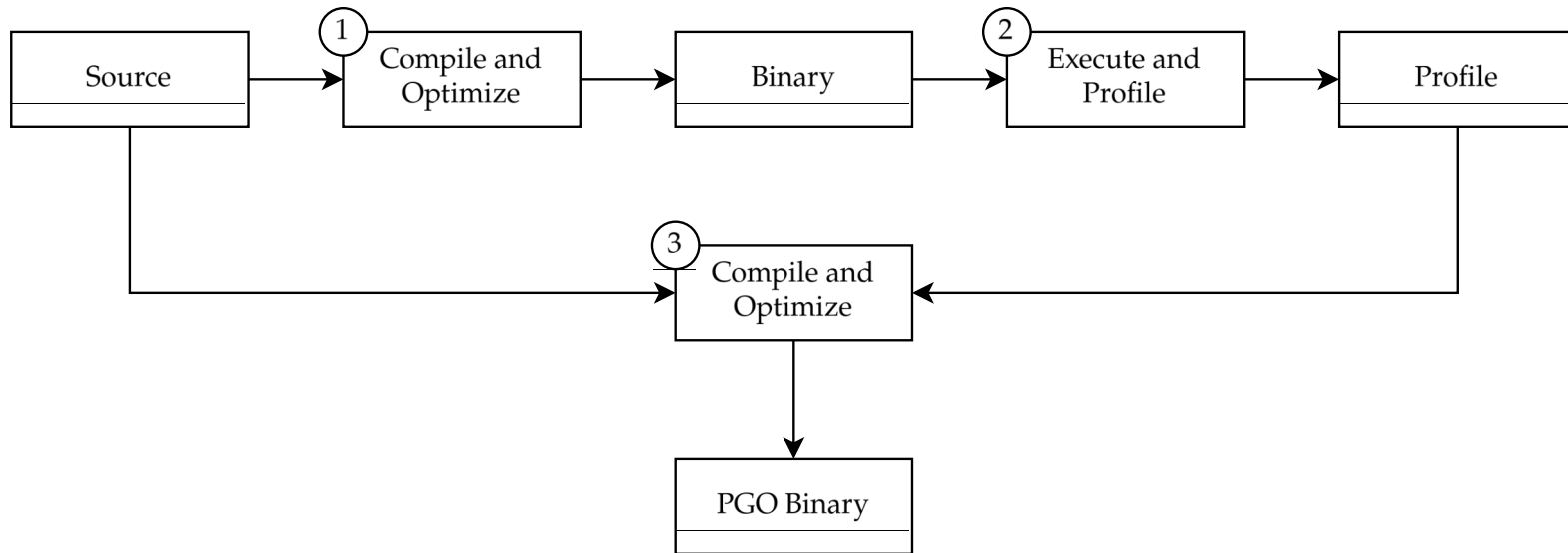


Profile Guided Optimization

- A smart optimization technique
 - Leverages *control flow information* captured at runtime to steer optimizations
 - Notable performance improvement
- Control flow information (or *profile*) a.k.a.
 - Weights on control flow edges
 - Counts on basic blocks
- Profiles can be captured via:
 - Instrumentation -> Additional logic inside the program
 - Sampling -> CPU counters + Perf
 - Tracing -> Profile aggregated from the program trace

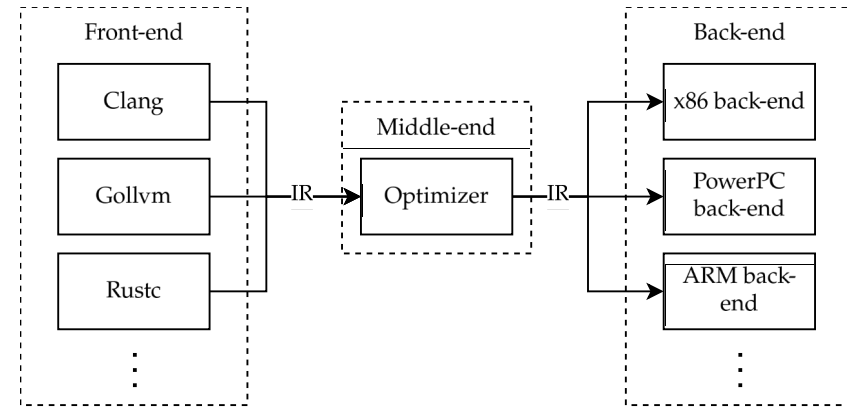


PGO Workflow



The LLVM Project

- The LLVM Project
 - A collection of modular and reusable compiler and toolchain technologies
 - Designed around a modern SSA-based compilation strategy

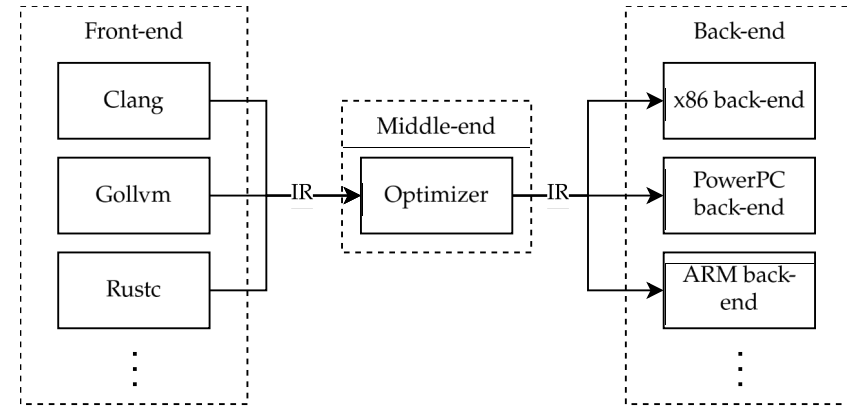


LLVM architecture



The LLVM Project

- The LLVM Project
 - A collection of modular and reusable compiler and toolchain technologies
 - Designed around a modern SSA-based compilation strategy
- Key Components
 - LLVM Core -> Source- and target-independent optimizer
 - Clang -> Native C/C++ compiler for LLVM
 - LLD -> High-performance linker



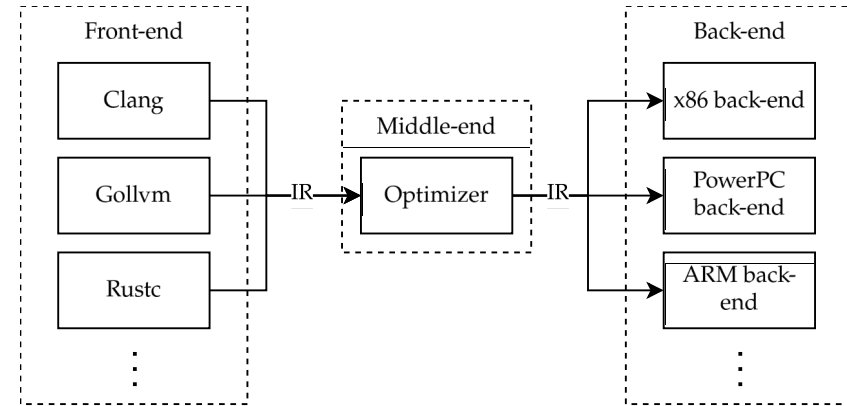
LLVM architecture



The LLVM Project

- The LLVM Project
 - A collection of modular and reusable compiler and toolchain technologies
 - Designed around a modern SSA-based compilation strategy
- Key Components
 - LLVM Core -> Source- and target-independent optimizer
 - Clang -> Native C/C++ compiler for LLVM
 - LLD -> High-performance linker

...and many more!



LLVM architecture



Profile Guided Optimization in LLVM

- Profile information encoded as instruction metadata
 - `branch_weights` -> Counts associated to instructions that change the control flow

```
bb41:  
  store i32 0, ptr %i40, align 4  
  %i42 = load <4 x i32>, ptr @h, align 16  
  br i1 %i39, label %.split.us.preheader, label %.split.preheader, !prof !30  
!30 = !{"branch_weights", i32 2000, i32 1000}
```

- `function_entry_counts` -> Number of times a function was called

```
define i32 @foo() !prof !1 {  
  ret i32 0  
}  
!1 = !{"function_entry_count", i64 2590}
```

- `vp` -> Profiles data values passed to functions (Not treated)



Profile Guided Optimization in LLVM

- Profile information encoded as instruction metadata
 - `branch_weights` -> Counts associated to instructions that change the control flow

```
bb41:  
  store i32 0, ptr %i40, align 4  
  %i42 = load <4 x i32>, ptr @h, align 16  
  br i1 %i39, label %.split.us.preheader, label %.split.preheader, !prof !30  
!30 = !{"branch_weights", i32 2000, i32 1000}
```

- `function_entry_counts` -> Number of times a function was called

```
define i32 @foo() !prof !1 {  
  ret i32 0  
}  
!1 = !{"function_entry_count", i64 2590}
```

- `vp` -> Profiles data values passed to functions (Not treated)



Profile Guided Optimization in LLVM

- Profile information encoded as instruction metadata
 - `branch_weights` -> Counts associated to instructions that change the control flow

```
bb41:  
  store i32 0, ptr %i40, align 4  
  %i42 = load <4 x i32>, ptr @h, align 16  
  br i1 %i39, label %.split.us.preheader, label %.split.preheader, !prof !30  
!30 = !{"branch_weights", i32 2000, i32 1000}
```

- `function_entry_counts` -> Number of times a function was called

```
define i32 @foo() !prof !1 {  
  ret i32 0  
}  
!1 = !{"function_entry_count", i64 2590}
```

- `vp` -> Profiles data values passed to functions (Not treated)



Control Flow Analysis

- Profile metadata used to compute additional control flow data
 - BranchProbabilityInfo -> Raw branch weights converted into branch probabilities
 - BlockFrequencyInfo -> Blocks are assigned an execution frequency relative to the entry block
 - ProfileSummaryInfo -> Hotness or coldness of blocks

```
entry:
    %cmp = icmp sgt i32 %x, 0
    br i1 %cmp, label %then, label %else, !prof !0

then:
    ret i32 1

else:
    ret i32 0

!0 = !{"branch_weights", i32 80, i32 20}
!1 = !{"function_entry_count", i64 1000}
```

BranchProbInfo

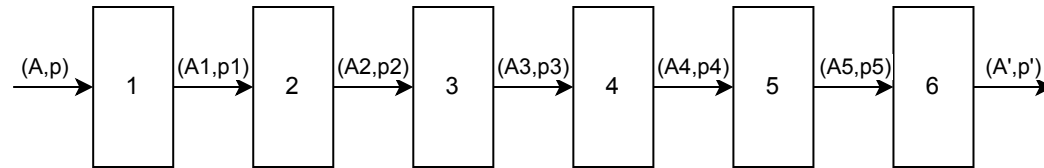
edge %entry → %then probability is $0 \times 66666666 / 0 \times 80000000 = 80.00\%$
 edge %entry → %else probability is $0 \times 19999999a / 0 \times 80000000 = 20.00\%$

BlockFrequencyInfo

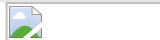
then: float = 0.8, int = 14411518804230144, count = 800
 else: float = 0.2, int = 3602879705251840, count = 200

Profile Information Propagation

- The starting profile is propagated throughout the entire pipeline
 - Each LLVM optimization is responsible for the update of the profile information
 - Additional logic to handle profile metadata in passes source code



- Bugs in such a logic reduce the efficacy of PGO
 - Subsequent pass will work on wrong profile information
 - A cascading effect will trigger
 - Bad optimization decisions can be taken!



Problem Formalization

Definition 3: Profile propagation analysis problem

Let O be a profile-guided optimization pipeline. Let (G, p) be a profiled program. Let $(G', p') = O(G, p)$ be the profiled program resulting by applying O to (G, p) . The *profile propagation analysis problem* consist of performing the following tasks:

- Determine if O made some profile propagation errors.
- If profile propagation errors were made by O , spot the faulty passes for this errors.



Profile and Block Frequency Formalization

Definition 1: Profiled Program

Given a program A represented as a control-flow graph $G = (E, V)$, a *profile* of A is a function $p : E \rightarrow \mathcal{N}$ that assigns to each edge $(v, w) \in G$ the number of times control flows from basic block v to basic block w . We denote the program A with profile p as the pair (A, p) .

Definition 2: Block Frequency

Give a profiled program $(G = (V, E), p)$ the *block frequency function* is a function $f_p : V \rightarrow \mathcal{N}$ that assigns to each basic block $v \in V$ the number of times v is reached during program execution, as derived from profile p .



Spotting profile propagation errors

Definition 4: Profile Equivalence Relation

Let p and q be two profiles for the same program G and let f_p and f_q be their respective block frequency functions. p is said to be equivalent to q if $\forall v \in V, f_p(v) = f_q(v)$



Spotting profile propagation errors

Definition 4: Profile Equivalence Relation

Let p and q be two profiles for the same program G and let f_p and f_q be their respective block frequency functions. p is said to be equivalent to q if $\forall v \in V, f_p(v) = f_q(v)$

- Now consider:



Spotting profile propagation errors

Definition 4: Profile Equivalence Relation

Let p and q be two profiles for the same program G and let f_p and f_q be their respective block frequency functions. p is said to be equivalent to q if $\forall v \in V, f_p(v) = f_q(v)$

- Now consider:
 - $p \rightarrow$ profile computed by the pipeline on the optimized program



Spotting profile propagation errors

Definition 4: Profile Equivalence Relation

Let p and q be two profiles for the same program G and let f_p and f_q be their respective block frequency functions. p is said to be equivalent to q if $\forall v \in V, f_p(v) = f_q(v)$

- Now consider:
 - p -> profile computed by the pipeline on the optimized program
 - q -> profile computed instrumenting and re-executing the optimized program



Spotting profile propagation errors

Definition 4: Profile Equivalence Relation

Let p and q be two profiles for the same program G and let f_p and f_q be their respective block frequency functions. p is said to be equivalent to q if $\forall v \in V, f_p(v) = f_q(v)$

- Now consider:
 - p -> profile computed by the pipeline on the optimized program
 - q -> profile computed instrumenting and re-executing the optimized program
 - if $p \neq q$ then pipeline made some profile propagation errors



Identifying Culprit Passes

Definition 5: Profile Mismatch

Let p and q be two profiles for the same program G and let f_p and f_q be their respective block frequency functions. Let $p \neq q$.

A *profile mismatch* is a tuple $(G, f, bb, f_p(bb), f_q(bb))$ such that

- f is a function within G
- bb is a basic block within f
- $f_p(bb) \neq f_q(bb)$



Identifying Culprit Passes

Definition 5: Profile Mismatch

Let p and q be two profiles for the same program G and let f_p and f_q be their respective block frequency functions. Let $p \neq q$.

A *profile mismatch* is a tuple $(G, f, bb, f_p(bb), f_q(bb))$ such that

- f is a function within G
 - bb is a basic block within f
 - $f_p(bb) \neq f_q(bb)$
-
- A single pipeline application can results in multiple profile mismatches



Identifying Culprit Passes

Definition 5: Profile Mismatch

Let p and q be two profiles for the same program G and let f_p and f_q be their respective block frequency functions. Let $p \neq q$.

A *profile mismatch* is a tuple $(G, f, bb, f_p(bb), f_q(bb))$ such that

- f is a function within G
 - bb is a basic block within f
 - $f_p(bb) \neq f_q(bb)$
-
- A single pipeline application can results in multiple profile mismatches
 - We need a way to attribute to a mismatch the pass that caused it...



Identifying Culprit Passes

Definition 6: Mismatch Equivalence Relation

Let G and G' be programs, where G' is obtained by manipulating G in some way. Let $m_1 = (G, f, bb, f_p(bb), f_q(bb))$ and $m_2 = (G', f', bb', f_p(bb'), f_q(bb'))$ be two mismatches for their respective programs. Then $m_1 = m_2$ if

- f and f' names are equal
- $f_p(bb) = f_p(bb')$
- $f_q(bb) = f_q(bb')$

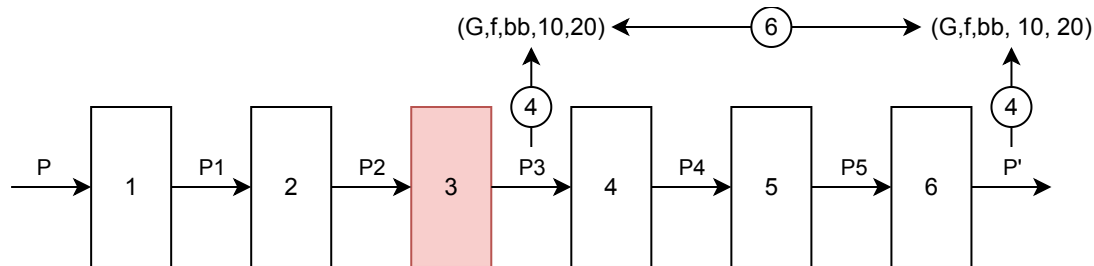


Identifying Culprit Passes

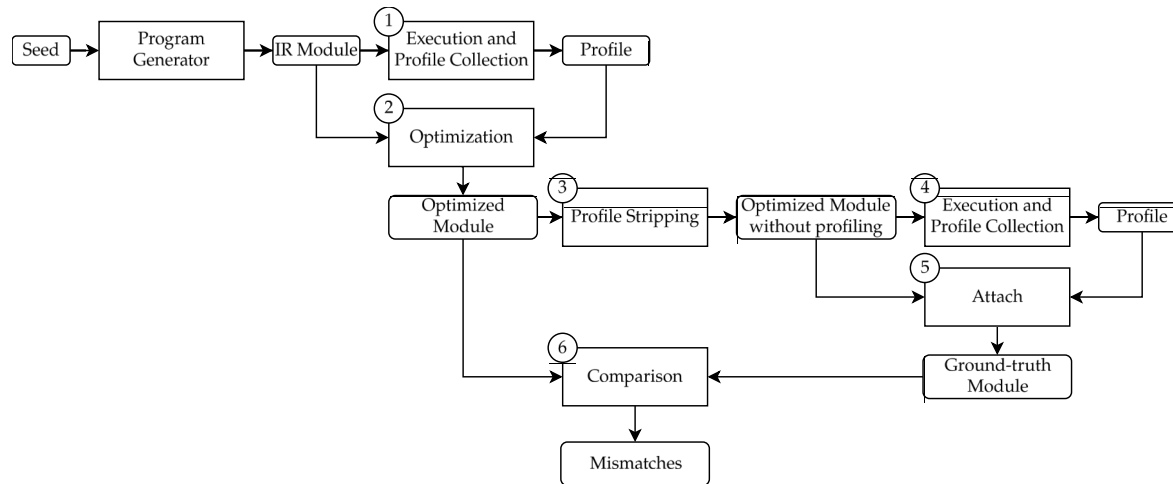
Definition 6: Mismatch Equivalence Relation

Let G and G' be programs, where G' is obtained by manipulating G in some way. Let $m_1 = (G, f, bb, f_p(bb), f_q(bb))$ and $m_2 = (G', f', bb', f_p(bb'), f_q(bb'))$ be two mismatches for their respective programs. Then $m_1 = m_2$ if

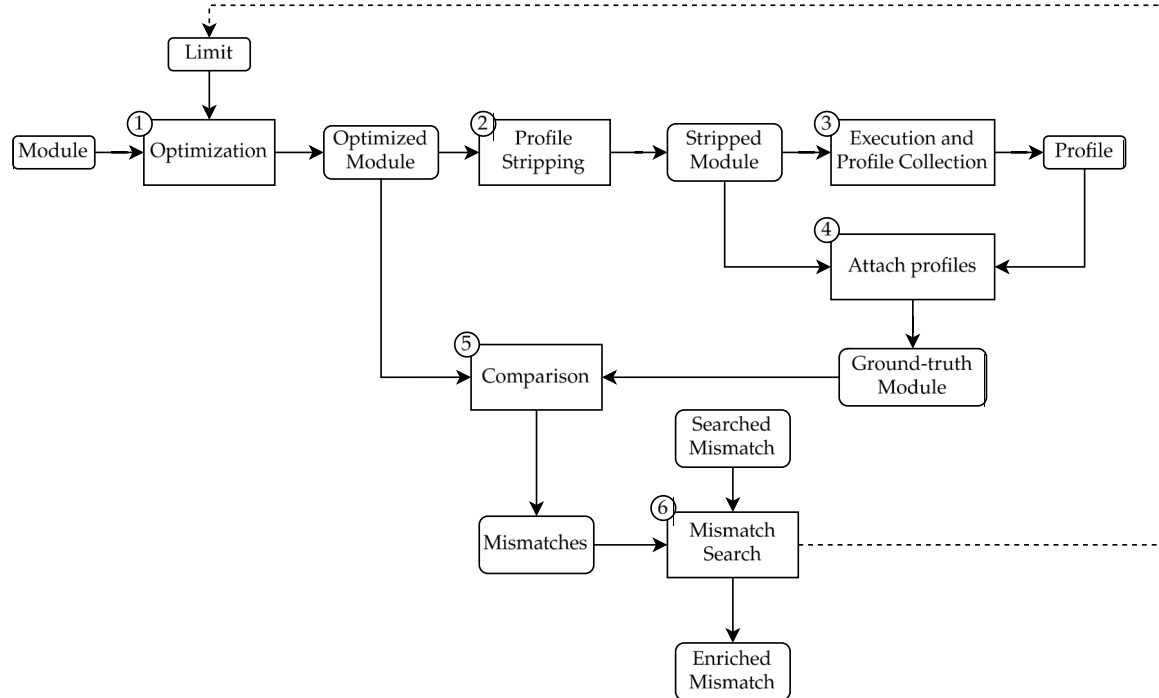
- f and f' names are equal
- $f_p(bb) = f_p(bb')$
- $f_q(bb) = f_q(bb')$



Optimization Phase



Search Phase



Results

- Eight major fuzzing campaigns were launched varying on
 - Optimization pipelines tested
 - Program generations parameters
 - Globally disabled passes
- Eleven total issues were found and reported to the LLVM community

Pass	Issues	Confirmed	Fixed
simplifycfg	3	1	1
pgo-instr-use	1	0	0
simple-loop-unswitch	1	0	0
chr	1	0	0
loop-rotate	1	0	0
jump-threading	1	1	1
loop-vectorize	1	1	0
globalopt	1	0	0
loop-unroll	1	0	0
Total	11	3	2

